Certify-or-Refuse: A Machine-Checked Soundness Boundary for Untrusted Agent Edits to Opaque-Semantics Artifacts

2026-07-07

Abstract

An LLM agent that edits a spreadsheet, a database schema, or a notebook produces a file that *opens fine* but may be silently wrong: a structural edit that fails to propagate references corrupts computed values with no visible symptom, and neither the agent nor a human reviewer can see it without the defining engine. We make the correctness of such an edit **checkable** rather than trusted. Our contribution is a formally verified soundness theorem, machine-checked in Lean 4 with no `sorry` and only the axioms `propext` and `Quot.sound`: evaluation of a computation is invariant under a function-and-dependency-preserving isomorphism, so a structural (coordinate- relabeling) edit whose reference-dependency graph is isomorphic to the original's reproduces every computed value — *under any semantics, without running the engine*. A companion locality theorem bounds what a value edit can affect to its downstream cone. On this spine we build a **certify-or-refuse router**: an untrusted agent's structural edit is accepted only when it equals the tool's own proven coordinate- shift transform, and otherwise explicitly refused — never silently wrong. We report a production certifier (`xlq certify`), the trusted reference-shift tokenizer it rests on (hardened to an exact grid-validity predicate and differentially cross-checked), and a fail-closed boundary for the one undecidable-from-syntax case (a defined name spelled like a cell reference). We are deliberate about what is *proof* and what is *corroboration*: the theorem is the result; the empirical harness — an engine-free foreign-edit certifier that refuses 147/147 corrupted edits, an independent-oracle A/B in which the naive edit path silently corrupts 86.6% of real workbooks while the certified path corrupts none, and an independent-oracle confusion matrix with zero observed false certifications — corroborates it, and we report each with its confound and its confidence interval, not as an independent soundness argument.

1. Introduction

The scarce resource in the agentic era is not capability but **verifiability**. A capable model will, with high probability, perform a structural spreadsheet edit correctly; but “high probability” is exactly the wrong guarantee for a financial model, a payroll sheet, or a regulatory filing, where a single unpropagated reference silently changes a computed total and ships. The failure is invisible by construction: the edited file is a valid `.xlsx`, it opens, and the wrong numbers sit in cells whose formulas *look* plausible. You cannot see the corruption without the engine, and asking the engine is asking the very component whose edit you are trying to check.

This paper asks a narrower, more durable question than "can the model do the edit?": **can we certify that a given edit is value-faithful, offline, against the artifact's own structure, without trusting the editor and without running the defining engine?** The answer we prove is yes for the *structural* fragment — the coordinate-relabeling edits (insert/delete rows and columns) that are simultaneously the most common agent operations and the ones whose damage is most invisible. The certifier accepts a structural edit only when its reference-dependency graph is the relabeling of the original's, and refuses otherwise. Because the guarantee is grounded in the artifact rather than the model, it does not decay as models improve — it is the boundary a capable agent should be *required* to pass, not a crutch for a weak one.

Our claims, in order of strength:

1. **(Proof.)** A machine-checked theorem (Lean 4, no `sorry`) that value-faithfulness of a structural edit reduces to graph isomorphism of its reference-dependency structure — engine-free and semantics-agnostic — plus a locality theorem bounding value-edit effects to a downstream cone (§3).
2. **(Mechanism.)** A certify-or-refuse router and a production certifier that decide accept/refuse for an untrusted foreign edit by comparing it to the tool's own proven transform, with a hardened trusted tokenizer and a fail-closed boundary for the one syntactically-undecidable case (§4).
3. **(Corroboration, reported with confounds.)** An engine-free foreign-edit certifier, an independent-oracle A/B, and an independent-oracle confusion matrix — each of which *supports* the theorem on real workbooks and each of which we report with its limitation, not as independent proof (§5).

A recurring, honest theme: three rounds of adversarial review of our own artifacts found real defects — silent-corruption bugs in the trusted tokenizer, a circular experiment, and an unimplemented soundness boundary — each of which we fixed and report as a fixed defect, because the discipline of finding them is part of the contribution (§6).

2. The problem: opaque-semantics artifacts and invisible structural damage

A spreadsheet is a program whose semantics live in an engine (Excel, LibreOffice Calc, IronCalc) that most tools do not reimplement. An edit tool that manipulates the file's bytes therefore edits a program it cannot evaluate. The dominant failure mode is the **unpropagated reference**: insert a row above a formula's input and, unless every reference below the insertion point is shifted, the formula now reads the wrong cell. The popular `openpyxl` library's `insert_rows` does exactly this — it moves cell contents but rewrites zero references — so on any workbook with a below-insertion reference, it produces a file that opens cleanly and computes wrong (§5.2).

The artifact carries its own partial ground truth: Excel writes a cached value <v> next to each formula. This *self-oracle* is what a certifier can check against — but only for cells the artifact chose to cache, and only if the checker can recompute, which returns us to the engine. Our theorem removes the recompute: for the structural fragment, value-faithfulness follows from structure alone.

3. The formal core (the contribution)

We model a computation as `C = (fn, deps)`: `fn n` maps a node's ordered dependency *values* to its value, and `deps n` is its ordered dependency *nodes*. This is exactly what a formula carries — what it reads and how it combines what it reads — with `fn` left an arbitrary (opaque) function. Evaluation is fuel-bounded: `eval C 0 _ = default` and `eval C (k+1) n = fn n (map (eval C k) (deps n))`. The theorem holds at every fuel level, hence for the true evaluation of any acyclic computation.

Theorem 1 (exact structural certification). Let `σ` map every node of `C` to a node of `C'` with the same function and dependency list relabeled by `σ` (a function- and edge-preserving graph isomorphism): `C'.fn (σ n) = C.fn n` and `C'.deps (σ n) = map σ (C.deps n)`. Then `eval C' k (σ n) = eval C k n` for all `k, n`.

Consequence. If the original's evaluation is the artifact's embedded ground truth `O` (`eval C k = O`, the self-oracle), then a structurally-faithful edit reproduces that ground truth at the relabeled positions, `eval C' k (σ n) = O n`, established without the defining engine and without recomputing `O`. This is the exact tier of certify-or-refuse.

Theorem 2 (locality / audit-surface bound). Define `agree_upto C C' k n`: the two computations have identical `fn` and `deps` at every node within `k` dependency-steps of `n`. Then `eval C k n = eval C' k n` — a node's value depends only on its dependency cone. Hence a value edit changes nothing outside its downstream cone: a mixed edit factors into a certified structural scaffold plus value fills whose effect is provably contained, collapsing the audit surface from "did this corrupt anything anywhere?" to "are these N cells right?".

Both theorems are machine-checked in Lean 4, self-contained (no Mathlib), with `#print axioms` reporting only `[propext, Quot.sound]` and no `sorry` (`formal/SelfOracle.lean`). The reference-shift algebra's arithmetic laws (insert∘delete = identity, monotonicity, the six-case delete clamp against the set-theoretic truth) are separately proved for all inputs by the Z3 SMT solver (`formal/shift_laws.py`).

What the theorem does and does not give. It gives: structural-edit value-fidelity reduces to a graph-isomorphism check that is engine-free and semantics-agnostic. It does *not* give: fidelity of value edits (out of scope by construction — routed to the probabilistic tier), nor a guarantee that the *extraction* of `(fn, deps)` from the concrete file is itself faithful. That extraction is the trusted computing base, and §4 is about making it small and checked rather than assumed.

4. The certify-or-refuse router and its trusted base

The router. Given an original artifact and an untrusted foreign edit plus a declared structural op, the router computes the op's node relabeling σ, and CERTIFIES iff the foreign edit's reference-dependency graph is exactly the σ-image of the original's; otherwise it REFUSES. Two properties make this sound as a check of *untrusted* work: under-declaring a change is caught (an unaccounted graph difference → refuse), and a wrong σ yields a mismatch → refuse. The router is fail-closed: any condition it cannot certify becomes a refusal.

The production certifier. `xlq certify <orig> <edited> --op … --at …` realizes the router with the tool's *complete* formula parser: it applies the tool's own proven structural transform to the original and diffs the result against the foreign edit, certifying iff the only differences are stripped/rewritten caches and number formats (which foreign tools routinely touch), and refusing on any formula, value, added, or removed-cell difference. It is engine-free (it compares stored formulas and raw data, never recomputed values) and multi-sheet-safe (it diffs the union of sheets). Certification therefore means "this foreign edit equals the tool's proven transform"; its marginal value over simply *doing* the edit is trust-topology — the untrusted producer's artifact is what ships, gated fail-closed, with the tool off the write path — not any verifier-cheaper-than-prover asymmetry (there is none; the certifier recomputes the transform).

The trusted base: the reference-shift tokenizer. Every certificate is only as sound as the predicate deciding *which tokens in a formula are cell references*. We initially used syntactic proxies (a 1–3-letter column, an unbounded row) and adversarial review found these were wrong in three ways that caused *silent corruption*: a function name whose prefix looks like a cell (`BIN2DEC` → `BIN3DEC` under a row insert), a function name ending in digits before a paren (`LOG10` → `LOG11`), and out-of-grid tokens (`XFE9`, `A2000000`) whose column or row exceeds the sheet limits. We replaced the proxies with the exact **grid-validity** predicate — a token is a cell reference iff its column is in `A..XFD` (1..16384) and its row is in `1..1048576`, boundary-gated so an identifier or function call is never mistaken for a reference. This one rule subsumes all four bug classes.

We validate the tokenizer by **differential cross-checking** against an independent A1 shifter over 19 digit-bearing Excel functions, out-of-grid tokens, $-absolutes, and ranges, across two operations (insert-rows and delete-rows): 175 (formula, op) pairs, 0 disagreements (`tokenizer_fuzz.py`). We are precise about the strength of this: the independent shifter is a second implementation of *our own* grid-validity spec, so 0 disagreements demonstrates **mechanization consistency** between the Rust scanner and the reference — not conformance to Excel's reference semantics, which our same-spec check cannot establish. Column operations, sheet-qualified/3D/table/R1C1 references, and unicode sheet names are not yet in the differential; they are covered only by same-spec unit tests. The honest label is *hardened and cross-checked, with the un-fuzzed construct classes stated as scope* — not *validated against ground truth*.

The one undecidable case, made a fail-closed boundary. A defined name spelled like a grid-valid cell (`FY2021` = column FY, row 2021; `Q1`; `Tax2020`) is indistinguishable from a reference *in the formula text*, so the tokenizer would shift its uses and the edited file would still equal the tool's own (wrong) transform — the single place `cer-`

`tified ⇒ correct` could be false on a real workbook. It is, however, decidable from the workbook's defined-names table, which the certifier already holds. We close it fail-closed: a structural edit on a workbook containing a defined name that collides with a cell-reference spelling emits a `defined_name_ref_collision` residual, and the certifier refuses. A workbook defining `FY2021` and using it in a formula routes to REFUSE; a benign name (`TaxRate`) proceeds. This converts the last hidden hole into a demonstrated boundary and scopes the verified surface explicitly: **single sheet, in-grid coordinates, row/column shifts, no name collision.**

5. Corroboration on real workbooks (reported with confounds)

None of the following is the soundness argument — that is §3. Each supports the theorem on real files and each carries a limitation we state.

5.1 Engine-free certification of untrusted foreign edits

Running the router (format-parametric core) on `openpyxl`'s edits of 172 real formula-bearing workbooks, engine-free, cross-checked against an independent LibreOffice oracle: **0 false certifications and 147/147 corrupted foreign edits refused** (`foreign_certify`). The load-bearing caveat, always reported alongside: this soundness is bought with heavy refusal — the fail-closed A1 proxy certifies only about **1 in 23** *faithful* foreign edits, because it refuses every reference form it cannot fully model (whole-row, whole-column, cross-sheet, table, defined-name). Useful soundness — a high certify rate on faithful edits — requires the complete parser of §4, not the proxy. The value is the boundary, not the throughput.

5.2 Independent-oracle A/B: the corruption the boundary prevents

On 172 real workbooks, insert-row@2, with LibreOffice (independent of both `openpyxl` and the tool's engine) recomputing each edit against Excel's cache: the naive `openpyxl` edit path **silently corrupts 86.6%** of workbooks (149/172), while the tool's structural edit is engine-confirmed faithful on 150/172 and explicitly refuses the remaining 22 — **0 silent corruptions** (`agent_ab`). We flag the confound: 86.6% is a corpus property × one known-library bug (openpyxl shifts no references), not an agent-error distribution, and a competent reference-shifting engine also gets this op right; the tool's differentiator is auditability and explicit refusal, which this A/B does not isolate. We found and fixed an oracle false-positive during this study — `ROW()/COLUMN()` are position-dependent and legitimately change on a row insert — which is why the oracle excludes position-dependent functions.

5.3 Independent-oracle confusion matrix for the production certifier

Adjudicating `xlq certify`'s verdicts by LibreOffice recompute (not by the tool), over 16 oracle-reliable workbooks (files where LibreOffice reproduces Excel's cache on the *unedited* original; ACCRINT/BESSEL-class engine disagreements are gated out so they cannot be miscounted): TP=14, **FN=0**, TN=13, FP=2 (`cert_confusion`). Three honest qualifications. First, `FN=0` on 14 corrupted edits is a *point estimate*; the rule-of-three 95% upper bound on the false-certification rate is $\approx 3/14 \approx 21\%$. Second, the corrupt arm is a monoculture — all 14 corruptions are `openpyxl`'s single deterministic no-op-shift, the maximally-detectable corruption — so the FN cell is satisfied, not stressed by adversarial partial-shift edits; and the certify verdict is confounded with editor identity (REFUSE ⇔ openpyxl, CERTIFY ⇔ tool). Third, the two FPs are on array/spill files where LibreOffice is itself blind to the corruption, so they are likely oracle blindness rather than over-conservatism — meaning the oracle has its own false negatives. The matrix establishes "the certifier refuses openpyxl's corruption and certifies the tool's transform"; it does not, by construction, exercise a self-consistent error, which is why §4's fail-closed name-collision boundary matters.

5.4 The interventional finding: differential testing hardened the trusted base

We ran a live fast-model agent on 20 real workbooks, tasking it to rewrite formula references for insert-row@2, and compared its output to the tool's transform. As a guard-soundness experiment this is **circular** — the "agent correct" label and the `xlq certify` verdict are the same predicate applied to a file grafted onto the tool's own output — and we do not report it as such. Its genuine, non-circular yield is **differential testing that surfaced real silent-corruption bugs in the tool's own tokenizer** (`BIN2DEC`, `Sales2020`, and on follow-up `LOG10` and the out-of-grid class), each now fixed and covered by §4's cross-check. A validation method that finds real bugs in its own trusted base is exactly what a certifier's TCB needs; the honest framing is that the method proved its worth, not that it produced a soundness number.

6. On finding our own defects

Three independent adversarial reviews of these artifacts each landed a real hole: the tokenizer's syntactic proxies (silent corruption), the circular live-agent experiment (a tautological "0 false certifications"), and the unimplemented defined-name boundary (a genuine open soundness gap). We fixed each and report it as fixed. We consider this part of the contribution: a certify-or-refuse claim is only credible if its authors have tried hardest to break it, and the record of what broke — and what the fix was — is the evidence that the remaining boundary is real.

7. Scope and limitations

The verified guarantee covers the structural fragment (row/column insert/delete) on single-sheet, in-grid coordinates with no defined-name collision; everything outside this surface routes to refusal, not silent acceptance. Value edits are out of the exact tier by construction. The tokenizer is hardened and cross-checked for mechanization consistency, not validated against Excel; column operations and sheet-qualified/3D/table/R1C1 constructs are not yet in the differential. The empirical corroboration is single-op (insert-row@2 for the certify eval), single oracle engine (LibreOffice, with its own array/spill blindness), and its corrupt arm is a one-tool monoculture — so its point estimates carry wide intervals. The exact tier certifies a measured 37.5% of operations and 27% of whole tasks on a realistic edit-distribution study; the majority of real tasks are mixed and need the probabilistic tier, whose soundness rests on the self-oracle's completeness rather than a proof.

8. Related work

Spreadsheet analysis and smell detection, structural-edit tools that reshape files without a fidelity property, and record/replay or unit-test approaches to spreadsheet correctness all differ from this work in the same way: they establish behavior on tested inputs, whereas we prove value-fidelity of the structural fragment for all inputs and all semantics, and gate untrusted edits on that proof. Verified compilation and translation validation are the closest methodological analogues — accept a producer's output only when it matches a proven-correct reference — which we adapt to the setting of opaque-semantics artifacts edited by untrusted agents.

9. Conclusion

Verifiability, not capability, is the durable constraint on agent edits to opaque-semantics artifacts. We proved — machine-checked, engine-free, semantics-agnostic — that value-fidelity of a structural spreadsheet edit reduces to a graph-isomorphism check, built a certify-or-refuse router that gates untrusted edits on that proof with a fail-closed boundary for the one undecidable case, and corroborated it on real workbooks while stating each confound. The result ships as an honest, genuinely-verified boundary over an explicitly-scoped surface — the boundary a capable agent should be required to pass, not a patch for a weak one.